# Redes Neuronales III

Caso de uso del sector salud · TensorFlow + Keras y PyTorch

Estudiante: Laura Fernanda Lopez Mayorga

Docente: Nidia Stella Garcia Roa

Deep Learning

Universidad de Cundinamarca

Octubre de 2026

# 1. Introducción y objetivos

Este documento presenta las tres prácticas desarrolladas en Google Colab durante las semanas 5, 6 y 7 del módulo de Deep Learning, aplicadas a un problema del sector salud: la clasificación de tumores de mama como benignos o malignos a partir de características medidas sobre imágenes de células obtenidas por punción con aguja fina. En la semana 5 se trabajan las técnicas de optimización, en la semana 6 la regularización y en la semana 7 las métricas de evaluación junto con el preprocesamiento y la distribución de los datos.

El diagnóstico asistido por modelos de aprendizaje profundo es un escenario especialmente adecuado para estudiar estas técnicas, porque los errores no tienen el mismo costo: un falso negativo (un tumor maligno clasificado como benigno) puede retrasar un tratamiento, mientras que un falso positivo genera exámenes adicionales. Por eso, además del accuracy, se analizan la sensibilidad, la especificidad y el ajuste del umbral de decisión.

### Objetivo general

Implementar, optimizar, regularizar y evaluar redes neuronales con TensorFlow + Keras y PyTorch sobre un caso del sector salud, documentando el proceso y los resultados obtenidos.

### Objetivos específicos

- Comprender los hiperparámetros y la función de los conjuntos train/dev/test.
- Aplicar y comparar Feature Scaling, Batch Normalization, Mini-batch Gradient Descent, Momentum, RMSProp, Adam y Learning Rate Decay.
- Diagnosticar bias y variance y reducir el overfitting con L1/L2, Dropout, Data Augmentation y Early Stopping.
- Evaluar el modelo con Accuracy, Precision, Recall, F1 Score, Sensibilidad, Especificidad, matriz de confusión y curva ROC.
- Publicar los notebooks y el proyecto en GitHub.

# 2. Caso de uso y datos

## 2.1 Planteamiento del problema

Dado un conjunto de mediciones de los núcleos celulares de una muestra de tejido mamario, el modelo debe predecir si el tumor es maligno (clase positiva, 1) o benigno (clase negativa, 0). Se trata de un problema de clasificación binaria sobre datos tabulares, por lo que se emplean perceptrones multicapa (MLP) con salida sigmoide y pérdida de entropía cruzada binaria.

## 2.2 Descripción del dataset

Se utilizó el dataset Breast Cancer Wisconsin (Diagnostic) (Wolberg et al., 1995), disponible en el repositorio UCI y distribuido con scikit-learn, por lo que se carga sin descargas externas en Google Colab. Contiene 569 muestras y 30 características numéricas, sin valores faltantes. Para cada núcleo celular se calculan diez medidas (radio, textura, perímetro, área, suavidad, compacidad, concavidad, puntos

cóncavos, simetría y dimensión fractal) y, de cada una, su media, su error estándar y su valor extremo ("worst"). Hay 357 casos benignos y 212 malignos (37,3 %), un desbalance moderado.

## 2.3 Distribución de los datos

Los datos se dividieron de forma estratificada (conservando la proporción de clases) con semilla fija para garantizar reproducibilidad.

| Conjunto | Proporción | Muestras | Malignos | % malignos |
|---|---|---|---|---|
| Train | 60 % | 341 | 127 | 37,2 % |
| Dev (validación) | 20 % | 114 | 43 | 37,7 % |
| Test | 20 % | 114 | 42 | 36,8 % |

*Tabla 1. Distribución estratificada de los datos.*

El conjunto de entrenamiento ajusta los pesos; el de desarrollo se usa para comparar configuraciones, elegir hiperparámetros, activar Early Stopping y definir el umbral de decisión; el de prueba se usa una única vez al final para estimar el desempeño sobre datos nunca vistos.

## 2.4 Preprocesamiento

Las 30 características tienen escalas muy distintas (por ejemplo, el área puede superar 2.000 mientras que la suavidad es menor a 0,2). Se aplicó estandarización (media 0 y desviación 1) ajustando el StandardScaler únicamente con el conjunto de entrenamiento y transformando después dev y test con esos mismos parámetros, lo que evita la fuga de datos (data leakage) desde los conjuntos de evaluación.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

SEED = 42

# 1. Carga: Breast Cancer Wisconsin (Diagnostic), incluido en scikit-learn
data = load_breast_cancer(as_frame=True)
X = data.data
y = (1 - data.target).astype("int32")      # 1 = maligno, 0 = benigno

# 2. Distribucion estratificada 60 / 20 / 20 (train / dev / test)
X_temp, X_test, y_temp, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=SEED)
X_train, X_dev, y_train, y_dev = train_test_split(
    X_temp, y_temp, test_size=0.25, stratify=y_temp, random_state=SEED)

# 3. Preprocesamiento: estandarizacion ajustada SOLO con train (evita fuga de datos)
scaler = StandardScaler().fit(X_train)
Xtr_s, Xdv_s, Xte_s = [scaler.transform(a).astype("float32")
                       for a in (X_train, X_dev, X_test)]
Xtr_raw = X_train.to_numpy(dtype="float32")     # sin escalar (solo para comparar)
Xdv_raw = X_dev.to_numpy(dtype="float32")
N_FEATURES = Xtr_s.shape[1]
```

```
resumen = pd.DataFrame({
    "conjunto": ["train", "dev", "test"],
    "muestras": [len(y_train), len(y_dev), len(y_test)],
    "malignos": [int(y_train.sum()), int(y_dev.sum()), int(y_test.sum())]})
resumen["% malignos"] = (100 * resumen["malignos"] / resumen["muestras"]).round(1)
print(resumen.to_string(index=False))
print("Caracteristicas:", N_FEATURES)
```

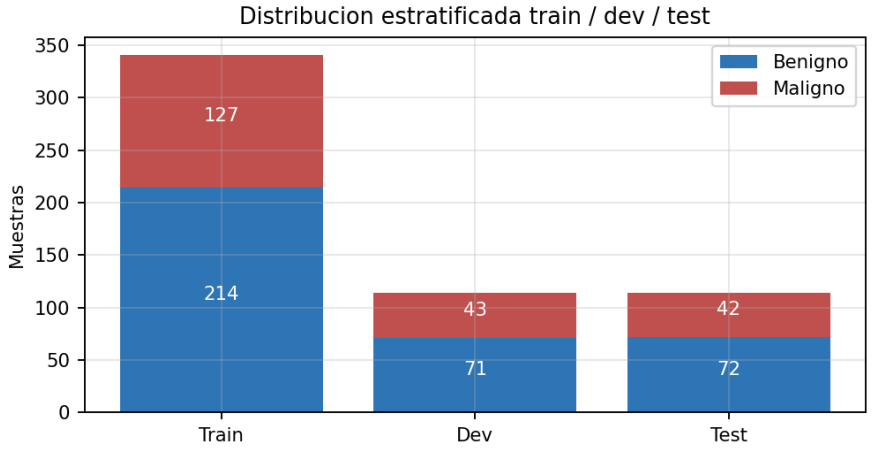



*Figura 1. Distribución de clases en los conjuntos train, dev y test.*

### 2.5 Nota sobre la ejecución de los experimentos

Los resultados de las secciones 4, 5 y 6 se obtuvieron ejecutando una implementación equivalente de la red en NumPy, con las mismas arquitecturas, la inicialización Glorot, la pérdida de entropía cruzada binaria, las fórmulas de los optimizadores (SGD, Momentum, RMSProp, Adam y decaimiento exponencial), Batch Normalization, Dropout, regularización L1/L2, ruido gaussiano, Early Stopping (paciencia de 15 épocas con restauración de los mejores pesos), la misma partición de datos y el mismo protocolo de tres semillas promediadas. La implementación se validó con una prueba numérica de gradientes (error relativo máximo inferior a 1e-7, incluyendo Batch Normalization y regularización). Los notebooks de TensorFlow + Keras que acompañan este informe aplican esa misma configuración; al ejecutarlos pueden aparecer diferencias menores en las cifras por la inicialización aleatoria y detalles de implementación, sin que cambien las conclusiones cualitativas. Los tiempos de cómputo corresponden a la implementación NumPy en CPU.

## 3. Marco conceptual

### 3.1 Hiperparámetros

Los hiperparámetros se fijan antes del entrenamiento y la red no los aprende, a diferencia de los pesos y sesgos que se ajustan por descenso del gradiente. Ejemplos: la tasa de aprendizaje (α), el tamaño del mini-batch, el número de épocas, la cantidad de capas y neuronas, el coeficiente de regularización (λ), la tasa de Dropout y los parámetros β1 y β2 de Adam. Se eligen de forma empírica evaluando sobre el conjunto dev, nunca sobre el de prueba.

## 3.2 Técnicas de optimización

| Técnica | Idea principal | Hiperparámetros clave |
|---|---|---|
| Feature Scaling | Normalizar las entradas para que el gradiente sea más estable y la convergencia más rápida. | Media y desviación (de train) |
| Batch Normalization | Normaliza las activaciones de cada capa por mini-batch y aprende escala (γ) y desplazamiento (β); estabiliza y acelera el entrenamiento. | momentum, ε |
| Mini-batch Gradient Descent | Actualiza los pesos con subconjuntos de datos: más rápido que el batch completo y menos ruidoso que el estocástico puro. | Tamaño del batch |
| Gradient Descent with Momentum | Acumula un promedio exponencial de los gradientes para amortiguar oscilaciones y acelerar en direcciones consistentes. | α, β (≈ 0,9) |
| RMSProp | Divide el gradiente por la raíz del promedio móvil de sus cuadrados, adaptando el paso por parámetro. | α, ρ (≈ 0,9) |
| Adam | Combina momentum (primer momento) y RMSProp (segundo momento) con corrección de sesgo. | α, β1 = 0,9, β2 = 0,999, ε |
| Learning Rate Decay | Reduce α durante el entrenamiento para afinar la convergencia cerca del mínimo. | Tasa y pasos de decaimiento |

## 3.3 Bias, variance y regularización

El bias (sesgo) es el error por suposiciones demasiado simples (underfitting): el error ya es alto en train. La variance es la sensibilidad del modelo a los datos de entrenamiento (overfitting): el error en train es bajo pero el de dev es notablemente mayor. El tradeoff consiste en que reducir uno suele aumentar el otro; se busca el punto de menor error en dev. Con solo 341 muestras de entrenamiento y 30 características, este dataset es propenso al overfitting, lo que lo hace adecuado para estudiar la regularización.

| Método | Funcionamiento | Parámetro |
|---|---|---|
| Regularización L2 | Penaliza la suma de cuadrados de los pesos; los mantiene pequeños. | λ (kernel_regularizer) |
| Regularización L1 | Penaliza el valor absoluto de los pesos; favorece pesos exactamente cero (modelos dispersos). | λ |
| Dropout | Apaga aleatoriamente un porcentaje de neuronas en cada paso de entrenamiento, evitando co-adaptaciones. | Tasa (0,2–0,5) |
| Data Augmentation | Amplía la diversidad del entrenamiento. En datos tabulares se aplica ruido gaussiano a las entradas (capa GaussianNoise) solo durante el entrenamiento. | Desviación del ruido |

| Método | Funcionamiento | Parámetro |
|---|---|---|
| Early Stopping | Detiene el entrenamiento cuando la pérdida en dev deja de mejorar y restaura los mejores pesos. | patience |

### 3.4 Ortogonalización y métricas

La ortogonalización propone separar los objetivos de mejora en "perillas" independientes: primero ajustar bien el conjunto de entrenamiento (red más grande, mejor optimizador), luego el de desarrollo (regularización, más datos), después el de prueba (conjunto dev más grande) y finalmente el desempeño en el mundo real. Así cada ajuste afecta un solo aspecto y el diagnóstico es más claro. En esta práctica la clase positiva es "maligno".

| Métrica | Definición | Lectura en el contexto clínico |
|---|---|---|
| Accuracy | (TP + TN) / total | Proporción de aciertos; puede ocultar errores graves si las clases están desbalanceadas. |
| Precision | TP / (TP + FP) | De los casos que el modelo marca como malignos, cuántos realmente lo son. |
| Recall / Sensibilidad | TP / (TP + FN) | De los casos malignos reales, cuántos detecta. Es la métrica crítica en tamizaje. |
| Especificidad | TN / (TN + FP) | De los casos benignos reales, cuántos se identifican como benignos. |
| F1 Score | 2·P·R / (P + R) | Media armónica entre Precision y Recall. |
| Matriz de confusión | Tabla real vs. predicho | Cuenta TN, FP, FN y TP; muestra el tipo de error. |
| AUC-ROC | Área bajo la curva TPR vs. FPR | Capacidad de separar las clases independientemente del umbral. |

### 3.5 TensorFlow + Keras y PyTorch

TensorFlow es una plataforma de cómputo numérico con diferenciación automática; Keras es su API de alto nivel para definir, entrenar y evaluar redes con pocas líneas (Sequential, compile, fit, evaluate). PyTorch usa grafos dinámicos y un estilo más explícito: el usuario escribe el ciclo de entrenamiento (forward, pérdida, backward, optimizer.step). Keras favorece el prototipado rápido y PyTorch ofrece más control; ambos usan GPU en Google Colab. Las prácticas se implementaron en Keras y se replicó el modelo final en PyTorch.

## 4. Práctica 1 – Técnicas de optimización (Semana 5)

Se construyó un MLP de dos capas ocultas (64 y 32 neuronas, ReLU) con salida sigmoide. Como el conjunto es pequeño y el resultado de una sola ejecución depende mucho de la inicialización, cada configuración

se entrenó con tres semillas (1, 2 y 3) y se reportó el promedio. Todas las comparaciones usan 100 épocas y mini-batch de 32, salvo cuando ese es el factor estudiado.

### 4.1 Modelo y utilidades (TensorFlow + Keras)

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, regularizers

keras.utils.set_random_seed(SEED)

def build_model(units=(64, 32), use_bn=False, dropout=0.0, l1=0.0, l2=0.0, noise=0.0):
    """MLP binario. noise>0 agrega ruido gaussiano (data augmentation tabular)."""
    reg = regularizers.L1L2(l1=l1, l2=l2) if (l1 or l2) else None
    m = keras.Sequential([layers.Input(shape=(N_FEATURES,))])
    if noise:
        m.add(layers.GaussianNoise(noise))
    for u in units:
        m.add(layers.Dense(u, use_bias=not use_bn, kernel_regularizer=reg))
        if use_bn:
            m.add(layers.BatchNormalization())
        m.add(layers.Activation("relu"))
        if dropout:
            m.add(layers.Dropout(dropout))
    m.add(layers.Dense(1, activation="sigmoid"))
    return m

def compile_model(m, optimizer):
    m.compile(optimizer=optimizer, loss="binary_crossentropy",
              metrics=["accuracy",
                       keras.metrics.Recall(name="recall"),
                       keras.metrics.AUC(name="auc")])
    return m

def entrenar(make_model, make_opt, Xtr=None, Xdv=None, epochs=100, batch_size=32,
             callbacks=None, seeds=(1, 2, 3)):
    """Entrena con varias semillas y promedia, porque el dataset es pequeno."""
    Xtr = Xtr_s if Xtr is None else Xtr
    Xdv = Xdv_s if Xdv is None else Xdv
    filas, hist = [], None
    for s in seeds:
        keras.backend.clear_session()
        keras.utils.set_random_seed(s)
        m = compile_model(make_model(), make_opt())
        h = m.fit(Xtr, y_train, validation_data=(Xdv, y_dev), epochs=epochs,
                  batch_size=batch_size, callbacks=callbacks() if callbacks else [],
                  verbose=0)
        tr = m.evaluate(Xtr, y_train, verbose=0, return_dict=True)
        dv = m.evaluate(Xdv, y_dev, verbose=0, return_dict=True)
        filas.append([tr["accuracy"], dv["accuracy"], dv["recall"], dv["auc"],
                      len(h.history["loss"])])
        hist = h.history
    r = np.array(filas)
    resumen = {"acc_train": r[:, 0].mean(), "acc_dev": r[:, 1].mean(),
               "std_dev": r[:, 1].std(), "recall_dev": r[:, 2].mean(),
               "auc_dev": r[:, 3].mean(), "epocas": r[:, 4].mean()}
    return resumen, hist
```

### 4.2 Feature Scaling

```
# A. Efecto del Feature Scaling (mismo modelo, mismo optimizador)
adam = lambda: keras.optimizers.Adam(1e-3)
res_raw, h_raw = entrenar(lambda: build_model(), adam, Xtr=Xtr_raw, Xdv=Xdv_raw)
```

```
res_esc, h_esc = entrenar(lambda: build_model(), adam)

print(pd.DataFrame({"Sin escalar": res_raw, "Estandarizado": res_esc}).round(4))

plt.figure(figsize=(7, 4))
plt.plot(h_raw["val_loss"], label="Sin escalar")
plt.plot(h_esc["val_loss"], label="Estandarizado")
plt.xlabel("Epoca"); plt.ylabel("Perdida en dev"); plt.legend(); plt.grid(alpha=.3)
plt.title("Efecto del Feature Scaling"); plt.show()
```

### 4.3 Comparación de optimizadores y Learning Rate Decay

```
# B. Comparacion de optimizadores (mini-batch = 32, 100 epocas, 3 semillas)
decay = keras.optimizers.schedules.ExponentialDecay(
    initial_learning_rate=1e-2, decay_steps=100, decay_rate=0.9)

optimizadores = {
    "SGD":              lambda: keras.optimizers.SGD(learning_rate=0.01),
    "SGD + Momentum":  lambda: keras.optimizers.SGD(learning_rate=0.01, momentum=0.9),
    "RMSProp":          lambda: keras.optimizers.RMSprop(learning_rate=1e-3),
    "Adam":             lambda: keras.optimizers.Adam(learning_rate=1e-3),
    "Adam + LR decay": lambda: keras.optimizers.Adam(learning_rate=decay),
}

tabla, curvas = {}, {}
for nombre, make_opt in optimizadores.items():
    tabla[nombre], curvas[nombre] = entrenar(lambda: build_model(), make_opt)
    print(f"{nombre:16s} acc_dev={tabla[nombre]['acc_dev']:.4f}
recall_dev={tabla[nombre]['recall_dev']:.4f}")

print(pd.DataFrame(tabla).T.round(4))

plt.figure(figsize=(8, 5))
for nombre, h in curvas.items():
    plt.plot(h["val_loss"], label=nombre)
plt.xlabel("Epoca"); plt.ylabel("Perdida en dev"); plt.legend(); plt.grid(alpha=.3)
plt.title("Comparacion de optimizadores"); plt.show()
```

### 4.4 Batch Normalization

```
# C. Efecto de Batch Normalization (Adam)
adam = lambda: keras.optimizers.Adam(1e-3)
res_sin, h_sin = entrenar(lambda: build_model(use_bn=False), adam)
res_con, h_con = entrenar(lambda: build_model(use_bn=True), adam)
print(pd.DataFrame({"Sin BatchNorm": res_sin, "Con BatchNorm": res_con}).round(4))

plt.figure(figsize=(7, 4))
plt.plot(h_sin["val_loss"], label="Sin BatchNorm")
plt.plot(h_con["val_loss"], label="Con BatchNorm")
plt.xlabel("Epoca"); plt.ylabel("Perdida en dev"); plt.legend(); plt.grid(alpha=.3)
plt.title("Efecto de Batch Normalization"); plt.show()
```

### 4.5 Tamaño del mini-batch

```
# D. Efecto del tamano del mini-batch (el ultimo equivale a batch gradient descent)
import time
filas = {}
for bs in (8, 32, 128, len(y_train)):
    t0 = time.time()
    r, _ = entrenar(lambda: build_model(), lambda: keras.optimizers.Adam(1e-3),
```

```
                    batch_size=bs)
    r["tiempo_s"] = time.time() - t0
    filas[f"batch={bs}"] = r
print(pd.DataFrame(filas).T.round(4))
```

### 4.6 Resultados

| Configuración | Acc. train | Acc. dev (± desv.) | Recall dev | AUC dev |
|---|---|---|---|---|
| Sin escalar | 92,3 % | 92,4 % ± 2,1 | 86,0 % | 0,989 |
| Estandarizado | 100,0 % | 96,8 % ± 0,4 | 97,7 % | 0,996 |
| SGD | 98,2 % | 97,4 % ± 0,0 | 96,1 % | 0,993 |
| SGD + Momentum | 100,0 % | 97,4 % ± 0,7 | 97,7 % | 0,995 |
| RMSProp | 100,0 % | 95,9 % ± 1,5 | 97,7 % | 0,996 |
| Adam | 100,0 % | 96,8 % ± 0,4 | 97,7 % | 0,996 |
| Adam + LR decay | 100,0 % | 96,8 % ± 1,1 | 97,7 % | 0,996 |
| Sin BatchNorm | 100,0 % | 96,8 % ± 0,4 | 97,7 % | 0,996 |
| Con BatchNorm | 100,0 % | 97,4 % ± 0,7 | 97,7 % | 0,993 |
| Batch 8 (3,2 s) | 100,0 % | 96,2 % ± 1,5 | 97,7 % | 0,996 |
| Batch 32 (1,4 s) | 100,0 % | 96,8 % ± 0,4 | 97,7 % | 0,996 |
| Batch 128 (0,8 s) | 100,0 % | 96,8 % ± 0,4 | 97,7 % | 0,995 |
| Batch 341 (1,0 s) | 98,8 % | 97,1 % ± 0,4 | 96,9 % | 0,995 |

*Tabla 2. Resultados de la Práctica 1: promedio de tres semillas sobre el conjunto dev (la desviación es entre semillas).*

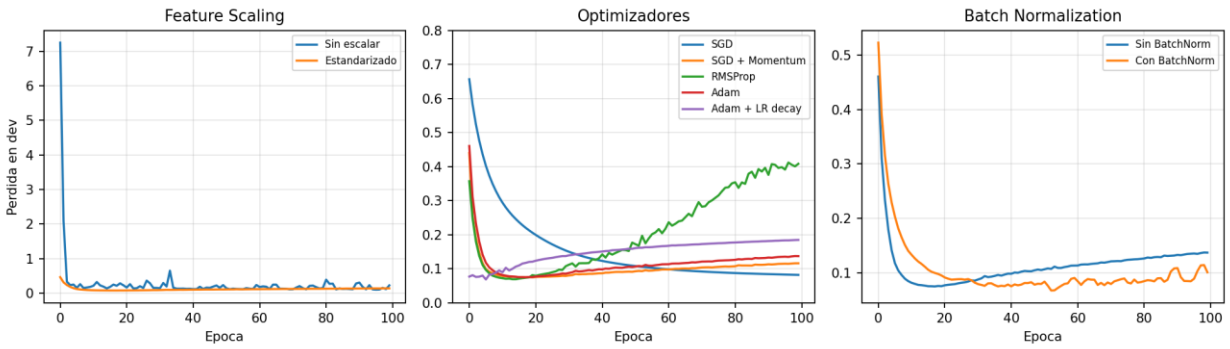



*Figura 2. Pérdida en dev por época: efecto del escalado, de los optimizadores y de Batch Normalization.*

### 4.7 Análisis

**Feature Scaling.** Fue el factor de mayor impacto. Sin escalar, el modelo solo alcanza 92,3 % en train y 92,4 % en dev (con una desviación de 2,1 puntos entre semillas) y una sensibilidad de 86,0 %; además, la pérdida en dev parte de un valor cercano a 7 y muestra picos (Figura 2, izquierda). Las características de gran magnitud, como el área y el perímetro, dominan el gradiente y dificultan la optimización (alto bias). Al estandarizar, el accuracy en dev sube a 96,8 % y la sensibilidad a 97,7 %.

**Optimizadores.** Con los datos estandarizados, todas las configuraciones quedaron entre 95,9 % y 97,4 % de accuracy en dev. Con 114 muestras, 1,5 puntos equivalen a menos de dos pacientes y son comparables a la desviación entre semillas (hasta 1,5 puntos), por lo que los datos no permiten afirmar que un optimizador sea mejor que otro en exactitud. Sí hay diferencias claras en la dinámica de entrenamiento: Adam, RMSProp y Momentum alcanzan su mínimo de pérdida en dev en unas 10 a 15 épocas y llegan a 100 % en train, mientras que SGD simple converge más lento (98,2 % en train tras 100 épocas). RMSProp es el que más rápido se sobreajusta: su pérdida en dev sube de 0,07 a cerca de 0,4. El Learning Rate Decay con tasa inicial de 1e-2 produce un arranque estable y una curva más suave.

**Batch Normalization.** Pasó de 96,8 % a 97,4 % en dev, una diferencia dentro del ruido; su efecto visible es que la pérdida en dev al final del entrenamiento es menor y crece menos (alrededor de 0,10 frente a 0,14), es decir, estabiliza el entrenamiento.

**Tamaño del mini-batch.** El accuracy fue similar entre tamaños (96,2 % a 97,1 %), pero el tiempo cambió: 3,2 s con batch de 8, 1,4 s con batch de 32, 0,8 s con batch de 128, 1,0 s con batch de 341 (tiempo de 100 épocas y tres semillas). El batch completo es el único que no ajusta totalmente el train en 100 épocas (98,8 %), porque realiza solo 100 actualizaciones frente a 1.100 con batch de 32.

Conclusión de la práctica: se adoptó Adam (tasa 1e-3) con datos estandarizados y mini-batch de 32 como configuración base, por su convergencia rápida y estable, y se tomó como señal clave que la pérdida en dev empieza a subir después de las primeras épocas, lo que motiva la regularización de la práctica siguiente.

# 5. Práctica 2 – Regularización (Semana 6)

Para provocar overfitting de forma controlada se entrenó una red grande (256-128-64 neuronas) sobre solo 341 muestras durante hasta 150 épocas. Luego se compararon L2, L1, Dropout, Data Augmentation (ruido gaussiano), Early Stopping y una combinación, con la misma arquitectura y el mismo optimizador (Adam). La utilidad de carga de datos y el modelo son los de las secciones 2.4 y 4.1.

## 5.1 Diagnóstico de bias y variance

| Escenario | Error train | Error dev | Diagnóstico y acción |
|---|---|---|---|
| A | Alto | Alto (similar) | Alto bias: red más grande, más épocas, mejor optimizador. |
| B | Bajo | Mucho mayor | Alta variance: regularizar, más datos, red más pequeña. |
| C | Alto | Aún mayor | Bias y variance altos: combinar ambas acciones. |
| D | Bajo | Bajo (similar) | Buen ajuste. |

## 5.2 Experimentos

```
# Modelo deliberadamente grande para provocar overfitting en un dataset pequeno
```

```
GRANDE = (256, 128, 64)
adam = lambda: keras.optimizers.Adam(1e-3)
early = lambda: [keras.callbacks.EarlyStopping(monitor="val_loss", patience=15,
                                               restore_best_weights=True)]

experimentos = {
    "Baseline (sin regularizar)": (dict(units=GRANDE), None),
    "L2 (1e-3)":                  (dict(units=GRANDE, l2=1e-3), None),
    "L1 (1e-4)":                  (dict(units=GRANDE, l1=1e-4), None),
    "Dropout (0.3)":              (dict(units=GRANDE, dropout=0.3), None),
    "Data augmentation (ruido)":  (dict(units=GRANDE, noise=0.1), None),
    "Early stopping":             (dict(units=GRANDE), early),
    "L2 + Dropout + Early stop.": (dict(units=GRANDE, l2=1e-3, dropout=0.3), early),
}

tabla, curvas = {}, {}
for nombre, (kw, cb) in experimentos.items():
    tabla[nombre], curvas[nombre] = entrenar(
        lambda kw=kw: build_model(**kw), adam, epochs=150, callbacks=cb)
    t = tabla[nombre]
    print(f"{nombre:28s} train={t['acc_train']:.4f} dev={t['acc_dev']:.4f} "
          f"brecha={t['acc_train']-t['acc_dev']:.4f} epocas={t['epocas']:.0f}")

df = pd.DataFrame(tabla).T
df["brecha"] = df["acc_train"] - df["acc_dev"]
print(df.round(4))
```

### 5.3 Curvas de pérdida

```
# Curvas de perdida: baseline vs. combinacion regularizada
fig, ax = plt.subplots(1, 2, figsize=(11, 4), sharey=True)
for a, nombre in zip(ax, ["Baseline (sin regularizar)", "L2 + Dropout + Early stop."]):
    a.plot(curvas[nombre]["loss"], label="train")
    a.plot(curvas[nombre]["val_loss"], label="dev")
    a.set_title(nombre); a.set_xlabel("Epoca"); a.legend(); a.grid(alpha=.3)
ax[0].set_ylabel("Perdida (binary cross-entropy)")
plt.tight_layout(); plt.show()
```

### 5.4 Resultados

| Método | Acc. train | Acc. dev | Brecha | Épocas |
|---|---|---|---|---|
| Baseline (sin regularizar) | 100,0 % | 97,1 % | 2,9 pts | 150 |
| L2 (1e-3) | 100,0 % | 97,7 % | 2,3 pts | 150 |
| L1 (1e-4) | 100,0 % | 95,6 % | 4,4 pts | 150 |
| Dropout (0.3) | 100,0 % | 96,8 % | 3,2 pts | 150 |
| Data augmentation (ruido) | 100,0 % | 95,9 % | 4,1 pts | 150 |
| Early stopping | 99,2 % | 97,1 % | 2,1 pts | 20 |
| L2 + Dropout + Early stopping | 100,0 % | 97,7 % | 2,3 pts | 85 |

*Tabla 3. Resultados de la Práctica 2: promedio de tres semillas (red 256-128-64, hasta 150 épocas).*

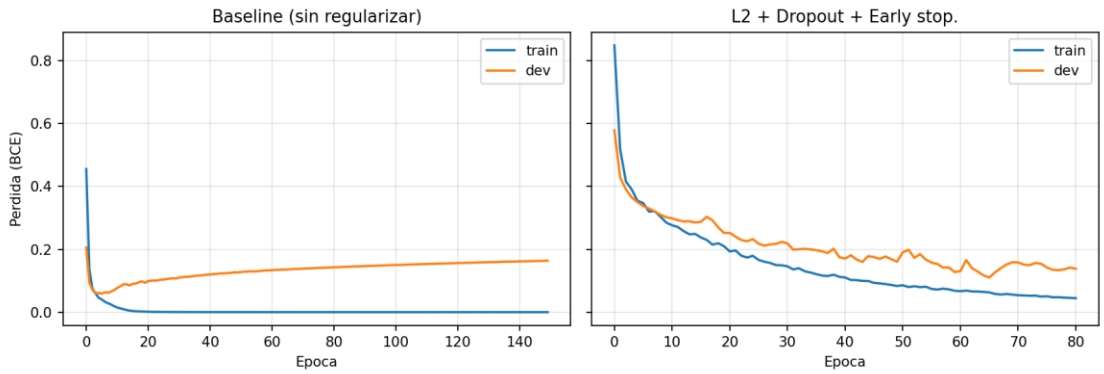



*Figura 3. Pérdida en train y dev del baseline y de la combinación regularizada (la pérdida del modelo regularizado incluye la penalización L2).*

### 5.5 Análisis

**Diagnóstico del baseline.** La red sin regularizar alcanza 100 % en train y 97,1 % en dev (brecha de 2,9 puntos). Su pérdida en dev toca un mínimo de aproximadamente 0,06 en las primeras épocas y luego sube de forma sostenida hasta cerca de 0,16, mientras la de train tiende a cero (Figura 3, izquierda): es el patrón de alta variance con bias bajo (escenario B).

**Efecto de cada método.** L2 (97,7 %) y la combinación L2 + Dropout + Early Stopping (97,7 %) fueron las mejores en dev y redujeron la brecha a 2,3 puntos. Sin embargo, la mejora sobre el baseline es de 0,6 puntos, menos de un paciente de dev, por lo que no es concluyente. Dropout (96,8 %), el ruido gaussiano (95,9 %) y L1 (95,6 %, con la mayor brecha: 4,4 puntos) no mejoraron la exactitud en dev con los valores de hiperparámetros probados. Early Stopping detuvo el entrenamiento en promedio a las 20 épocas de 150 con 97,1 % en dev, igual que el baseline pero con unas siete veces menos cómputo; su sensibilidad en dev fue algo menor (96,1 % frente a 97,7 %), equivalente a un paciente.

**Selección.** Se eligió la combinación L2 + Dropout + Early Stopping para el modelo final: empata en la mayor exactitud en dev, tiene la menor variabilidad entre semillas (± 0,4 puntos frente a ± 0,8 de L2 sola) y entrena menos épocas (85 en promedio). Con tan pocos datos, las diferencias entre métodos son pequeñas y la elección se apoya más en la estabilidad y el costo de cómputo que en la exactitud. Como nota de lectura, la pérdida de los modelos con L2 incluye el término de penalización, por lo que su valor absoluto no es directamente comparable con el del baseline.

## 6. Práctica 3 – Métricas de evaluación (Semana 7)

Con la configuración regularizada seleccionada (MLP 64-32, L2 = 1e-3, Dropout = 0,3, Adam y Early Stopping) se entrenó el modelo final con train, se validó con dev y se evaluó una sola vez sobre test. Se calcularon Accuracy, Precision, Recall (sensibilidad), especificidad, F1, AUC y la matriz de confusión.

### 6.1 Modelo final y métricas

```
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             confusion_matrix, ConfusionMatrixDisplay, roc_curve,
                             roc_auc_score, classification_report)
```

```
# Modelo final: L2 + Dropout + Early Stopping (entrenado con train, validado con dev)
keras.backend.clear_session()
keras.utils.set_random_seed(SEED)
final = compile_model(build_model(units=(64, 32), dropout=0.3, l2=1e-3),
                      keras.optimizers.Adam(1e-3))
es = keras.callbacks.EarlyStopping(monitor="val_loss", patience=15,
                                   restore_best_weights=True)
hist = final.fit(Xtr_s, y_train, validation_data=(Xdv_s, y_dev), epochs=200,
                 batch_size=32, callbacks=[es], verbose=0)
print("Epocas entrenadas:", len(hist.history["loss"]))

p_dev = final.predict(Xdv_s, verbose=0).ravel()    # probabilidad de malignidad
p_test = final.predict(Xte_s, verbose=0).ravel()

def metricas(y_true, p, umbral=0.5):
    y_pred = (p >= umbral).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
    return {"accuracy": accuracy_score(y_true, y_pred),
            "precision": precision_score(y_true, y_pred, zero_division=0),
            "recall (sensibilidad)": recall_score(y_true, y_pred),
            "especificidad": tn / (tn + fp),
            "f1": f1_score(y_true, y_pred),
            "auc": roc_auc_score(y_true, p),
            "TN": tn, "FP": fp, "FN": fn, "TP": tp}

m_dev, m_test = metricas(y_dev, p_dev), metricas(y_test, p_test)
print(pd.DataFrame({"dev": m_dev, "test": m_test}).round(4))
```

### 6.2 Matriz de confusión y curva ROC

```
# Matriz de confusion y reporte por clase (test, umbral 0.5)
y_pred = (p_test >= 0.5).astype(int)
print(classification_report(y_test, y_pred, target_names=["Benigno", "Maligno"],
digits=4))

ConfusionMatrixDisplay(confusion_matrix(y_test, y_pred),
                       display_labels=["Benigno", "Maligno"]).plot(cmap="Blues",
values_format="d")
plt.title("Matriz de confusion - test (umbral 0.5)"); plt.show()

# Curva ROC
fpr, tpr, _ = roc_curve(y_test, p_test)
plt.figure(figsize=(5, 5))
plt.plot(fpr, tpr, label=f"AUC = {roc_auc_score(y_test, p_test):.3f}")
plt.plot([0, 1], [0, 1], "--", color="gray")
plt.xlabel("1 - Especificidad (FPR)"); plt.ylabel("Sensibilidad (TPR)")
plt.title("Curva ROC - test"); plt.legend(); plt.grid(alpha=.3); plt.show()
```

### 6.3 Ajuste del umbral orientado a sensibilidad

En tamizaje oncológico suele preferirse una sensibilidad alta, aun a costa de más falsos positivos, porque un falso negativo es más costoso. El umbral de decisión se ajusta con el conjunto dev (nunca con test). Como dev tiene pocas muestras, no se toma el umbral extremo que cumple el objetivo, sino el punto medio del rango de umbrales que alcanzan una sensibilidad de al menos 98 % en dev, para reducir la dependencia de casos individuales.

```
# Barrido de umbrales (la salida del modelo es una probabilidad de malignidad)
filas = []
for u in (0.01, 0.05, 0.1, 0.2, 0.3, 0.5, 0.7, 0.9):
```

```
    for nombre, yt, pp in (("dev", y_dev, p_dev), ("test", y_test, p_test)):
        tn, fp, fn, tp = confusion_matrix(yt, (pp >= u).astype(int)).ravel()
        filas.append({"umbral": u, "conjunto": nombre, "sensibilidad": tp / (tp + fn),
                      "especificidad": tn / (tn + fp), "FN": fn, "FP": fp})
print(pd.DataFrame(filas).round(3).to_string(index=False))

# Ajuste del umbral con DEV: en tamizaje interesa no perder casos malignos (FN costoso).
# Objetivo: sensibilidad >= 98 % en dev. Se toma el punto medio del rango de umbrales que
# lo cumplen (no el borde), porque con ~114 muestras en dev el borde es muy inestable.
umbrales = np.linspace(0.01, 0.99, 99)
admisibles = [u for u in umbrales if recall_score(y_dev, p_dev >= u) >= 0.98]
umbral = float(np.median(admisibles)) if admisibles else 0.5
print("Rango admisible en dev:", (round(min(admisibles), 2), round(max(admisibles), 2)) if
admisibles else "ninguno")
print("Umbral elegido:", round(umbral, 3))

m_test_u = metricas(y_test, p_test, umbral)
comparacion = pd.DataFrame({"test (umbral 0.5)": m_test,
                            f"test (umbral {umbral:.3f})": m_test_u})
print(comparacion.round(4))
```

## 6.4 Resultados

| Métrica | Dev (umbral 0,5) | Test (umbral 0,5) | Test (umbral 0,015) |
|---|---|---|---|
| Accuracy | 98,2 % | 97,4 % | 91,2 % |
| Precision | 97,7 % | 100,0 % | 82,0 % |
| Recall (sensibilidad) | 97,7 % | 92,9 % | 97,6 % |
| Especificidad | 98,6 % | 100,0 % | 87,5 % |
| F1 Score | 97,7 % | 96,3 % | 89,1 % |
| AUC-ROC | 0,996 | 0,989 | 0,989 |
| TN / FP / FN / TP | 70 / 1 / 1 / 42 | 72 / 0 / 3 / 39 | 63 / 9 / 1 / 41 |

*Tabla 4. Métricas del modelo final (entrenado 110 épocas hasta Early Stopping). Clase positiva: maligno.*

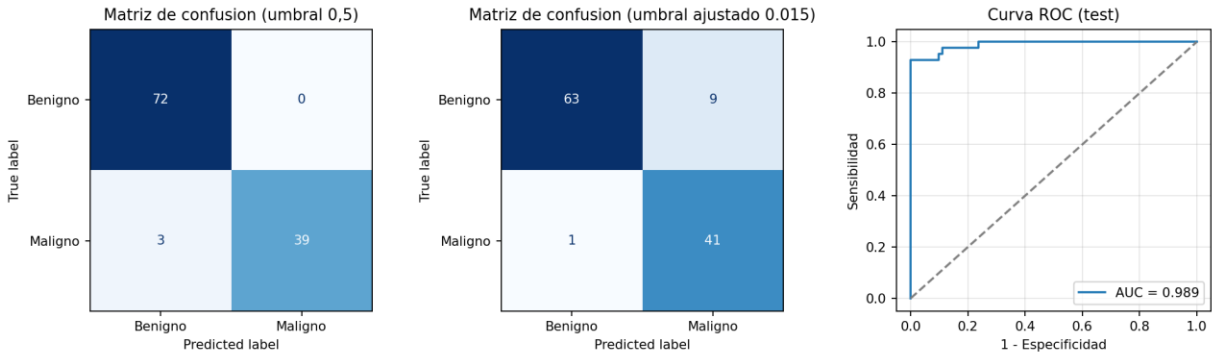



*Figura 4. Matrices de confusión en test con dos umbrales y curva ROC.*

Barrido de umbrales (la salida del modelo es la probabilidad de malignidad):

| Umbral | Sens. dev | Esp. dev | Sens. test | FN test | FP test |
|---|---|---|---|---|---|
| 0,01 | 100,0 % | 81,7 % | 97,6 % | 1 | 9 |
| 0,05 | 97,7 % | 87,3 % | 95,2 % | 2 | 8 |
| 0,10 | 97,7 % | 88,7 % | 92,9 % | 3 | 6 |
| 0,20 | 97,7 % | 93,0 % | 92,9 % | 3 | 5 |
| 0,30 | 97,7 % | 94,4 % | 92,9 % | 3 | 1 |
| 0,50 | 97,7 % | 98,6 % | 92,9 % | 3 | 0 |
| 0,70 | 97,7 % | 98,6 % | 92,9 % | 3 | 0 |
| 0,90 | 97,7 % | 100,0 % | 90,5 % | 4 | 0 |

*Tabla 5. Sensibilidad, especificidad y errores según el umbral de decisión.*

### 6.5 Análisis

**Desempeño con umbral 0,5.** En test el modelo obtuvo accuracy de 97,4 %, precision de 100 %, sensibilidad de 92,9 %, especificidad de 100 %, F1 de 96,3 % y AUC de 0,989 (72 verdaderos negativos, 0 falsos positivos, 3 falsos negativos y 39 verdaderos positivos). No hubo falsos positivos, pero 3 de los 42 tumores malignos se clasificaron como benignos, que es el error más costoso en tamizaje. La clase benigna tuvo recall de 100 % y precision de 96,0 %; la maligna, precision de 100 % y recall de 92,9 %.

**Sobreconfianza.** Las probabilidades de malignidad asignadas a esos tres casos fueron 0,001, 0,031 y 0,055: el modelo estaba muy seguro de su error. Por eso, entre los umbrales 0,3 y 0,7 el desempeño casi no cambia (Tabla 5) y sugiere que, antes de cualquier uso real, habría que calibrar las probabilidades. La sensibilidad pasó de 97,7 % en dev (1 falso negativo) a 92,9 % en test (3 falsos negativos); con solo 42 casos malignos en test, cada falso negativo mueve la sensibilidad cerca de 2,4 puntos, así que estas cifras tienen una incertidumbre considerable.

**Ajuste del umbral.** Con el objetivo de sensibilidad de al menos 98 % en dev, el rango de umbrales admisibles fue de 0,01 a 0,02 y se eligió el punto medio, 0,015. En test, la sensibilidad subió a 97,6 % (los falsos negativos pasaron de 3 a 1), pero la especificidad bajó a 87,5 % (los falsos positivos pasaron de 0 a 9), con precision de 82,0 %, accuracy de 91,2 % y F1 de 89,1 %. Es decir, detectar dos tumores malignos adicionales costó nueve falsos positivos. El AUC no cambia (0,989) porque no depende del umbral. Qué umbral usar es una decisión clínica sobre el costo relativo de cada error: en tamizaje suele aceptarse más falsos positivos si se confirman luego con otras pruebas.

### 6.6 Consideraciones éticas y limitaciones

- El dataset es pequeño y proviene de una única fuente; el modelo no está validado en otras poblaciones ni equipos de captura.
- Las características son medidas derivadas de imágenes, no las imágenes originales; el modelo no reemplaza el criterio del patólogo.
- Un modelo de apoyo diagnóstico requeriría validación externa, calibración de probabilidades, análisis de sesgos y aprobación regulatoria antes de su uso clínico.

# 7. Anexo – Equivalente en PyTorch

Se replicó el modelo final en PyTorch con los mismos datos (celda de la sección 2.4): BatchNorm, Dropout, Adam con weight decay (L2 acoplada), decaimiento escalonado de la tasa de aprendizaje (StepLR) y Early Stopping implementado manualmente, para ilustrar que en PyTorch el ciclo de entrenamiento se escribe de forma explícita.

```
import torch, torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix)

device = "cuda" if torch.cuda.is_available() else "cpu"
torch.manual_seed(SEED)

def loader(X, y, shuffle):
    ds = TensorDataset(torch.tensor(X), torch.tensor(y.to_numpy(), dtype=torch.float32))
    return DataLoader(ds, batch_size=32, shuffle=shuffle)

train_dl, dev_dl = loader(Xtr_s, y_train, True), loader(Xdv_s, y_dev, False)

model = nn.Sequential(
    nn.Linear(N_FEATURES, 64), nn.BatchNorm1d(64), nn.ReLU(), nn.Dropout(0.3),
    nn.Linear(64, 32), nn.BatchNorm1d(32), nn.ReLU(), nn.Dropout(0.3),
    nn.Linear(32, 1)).to(device)                    # logits (sigmoide implicita en la
perdida)

loss_fn = nn.BCEWithLogitsLoss()
opt = torch.optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-3)    # L2 acoplada
sched = torch.optim.lr_scheduler.StepLR(opt, step_size=30, gamma=0.5)     # LR decay

mejor, paciencia, esperando, mejores_pesos = float("inf"), 15, 0, None
for epoca in range(200):
    model.train()
    for xb, yb in train_dl:
        xb, yb = xb.to(device), yb.to(device)
        opt.zero_grad()
        loss = loss_fn(model(xb).squeeze(1), yb)
        loss.backward()
        opt.step()
    sched.step()

    model.eval(); val = 0.0
    with torch.no_grad():
        for xb, yb in dev_dl:
            val += loss_fn(model(xb.to(device)).squeeze(1), yb.to(device)).item() *
len(yb)
    val /= len(dev_dl.dataset)

    if val < mejor:                                 # Early stopping manual
        mejor, esperando = val, 0
        mejores_pesos = {k: v.clone() for k, v in model.state_dict().items()}
    else:
        esperando += 1
        if esperando >= paciencia:
            print("Early stopping en la epoca", epoca + 1)
            break

model.load_state_dict(mejores_pesos); model.eval()
with torch.no_grad():
```

```
    p = torch.sigmoid(model(torch.tensor(Xte_s).to(device)).squeeze(1)).cpu().numpy()
y_pred = (p >= 0.5).astype(int)
tn, fp, fn, tp = confusion_matrix(y_test, y_pred).ravel()
print(f"accuracy={accuracy_score(y_test, y_pred):.4f}  precision={precision_score(y_test,
y_pred):.4f}  "
      f"recall={recall_score(y_test, y_pred):.4f}  especificidad={tn/(tn+fp):.4f}
f1={f1_score(y_test, y_pred):.4f}")
print("Matriz de confusion [[TN, FP], [FN, TP]]:", [[tn, fp], [fn, tp]])
```

## 8. Conclusiones

- Una distribución train/dev/test estratificada y un preprocesamiento ajustado solo con train son la base de una evaluación honesta; el conjunto de prueba se usó una sola vez para reportar el modelo final.
- El Feature Scaling fue la mejora más importante: el accuracy en dev pasó de 92,4 % a 96,8 % y la sensibilidad de 86,0 % a 97,7 %. Con los datos estandarizados, los cinco optimizadores quedaron entre 95,9 % y 97,4 % en dev, diferencias dentro del ruido de un conjunto de 114 muestras; difieren más en la velocidad de convergencia y en el sobreajuste (RMSProp fue el más rápido en sobreajustarse).
- El baseline mostró alta variance (100 % en train, 97,1 % en dev, y pérdida en dev creciente desde las primeras épocas). L2 y la combinación L2 + Dropout + Early Stopping dieron la mejor exactitud en dev (97,7 %), y Early Stopping redujo el cómputo unas siete veces sin perder exactitud; las mejoras de exactitud son pequeñas y no concluyentes con tan pocos datos.
- El modelo final alcanzó en test 97,4 % de accuracy, 100 % de precision, 92,9 % de sensibilidad y AUC de 0,989, pero con 3 falsos negativos a los que asignó probabilidades muy bajas, lo que revela sobreconfianza y la necesidad de calibrar el modelo.
- En salud el accuracy no basta: bajar el umbral a 0,015 elevó la sensibilidad a 97,6 % pero redujo la especificidad a 87,5 %, un intercambio que debe decidirse según el costo clínico de cada tipo de error.
- Limitaciones: dataset pequeño y de una sola fuente, sin validación externa, resultados con incertidumbre alta por el tamaño de dev y test, y uso estrictamente académico.

## 9. Repositorio en GitHub

Enlace de colab 1 :
https://colab.research.google.com/drive/1hbcE3z4zZ1avVJt9U6YZt9QERoXFPm_f?usp=sharing

Enlace Colab 2: https://colab.research.google.com/drive/1ZnUQogqkt003SksJy7jxv-w3Z3ZHZJnZ?usp=sharing

Enlace Colab 3:
https://colab.research.google.com/drive/1H8_PsWshQCwtrj1LKBtGBCsajnRCBEiN?usp=sharing

Enlace Colab 4: https://colab.research.google.com/drive/1VK-v0y-JZpOM-dqGnuaSMkDR9zgJB7di?usp=sharing

Enlace de Github: https://github.com/lFernandaalopez/Red-neuronal-III.git

# 10. Referencias

Deep Learning with PyTorch. (2020). Deep learning with PyTorch. Manning Publications. https://isip.piconepress.com/courses/temple/ece_4822/resources/books/Deep-Learning-with-PyTorch.pdf

Deep Learning with TensorFlow and Keras. (2023). Deep learning with TensorFlow and Keras (3rd ed.). Packt Publishing. https://download.packt.com/free-ebook/9781803232911

GitHub. (2023). Deep Learning with TensorFlow and Keras – 3rd edition repository. https://github.com/PacktPublishing/Deep-Learning-with-TensorFlow-and-Keras-3rd-edition

GitHub. (2026). GitHub. https://github.com/

Google. (2026). Google Colaboratory. Google Research. https://colab.research.google.com/

Grokking Deep Learning. (2019). Grokking deep learning. Manning Publications. https://edu.anarcho-copy.org/Algorithm/grokking-deep-learning.pdf

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., … Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Wolberg, W. H., Street, W. N., & Mangasarian, O. L. (1995). Breast Cancer Wisconsin (Diagnostic) [Conjunto de datos]. UCI Machine Learning Repository. https://doi.org/10.24432/C5DW2B

YouTube. (2020). ¿Qué es una red neuronal? | Aprendizaje profundo. Capítulo 1 [Video]. YouTube.